MTE / Scudo Research — Pre-Meeting Handout

Prepared for internal discussion. Covers mitigation mechanics, the defensive research thesis, verified platform facts, and the authorization path. Scope note: this document reasons about how the mitigation works and how the research should be authorized and disclosed — it does not contain exploit construction.

Topic 1 — How MTE tags work (the core concept)

Q: What is a pointer tag? Every MTE-protected pointer carries a 4-bit tag in its top byte. The memory it points to is "painted" with a matching tag. On each dereference, the hardware checks that the pointer's tag matches the memory's tag. Mismatch → fault.

Q: What is a "known" vs "unknown" tag, and why does it matter? From an attacker's point of view, a tag is *known* if its value has been learned and *unknown* if it would have to be guessed. Guessing is roughly 1-in-15 (tag 0 is excluded). This distinction is the entire hinge of MTE's protection: acting on a pointer whose tag you don't know is a coin-flip that usually faults. **MTE's security guarantee rests on tags staying unknown.** That single sentence is the thesis of this research (see Topic 3).

Topic 2 — How Scudo implements tagging

Q: Does Scudo tag whole objects? Yes. One tag covers the entire allocation — a single tag value is written across the whole chunk, so every 16-byte granule shares one color. *Consequence:* MTE cannot see an overflow that stays **inside** one allocation (one field into another of the same object); it only catches an access that crosses **out** into a differently-tagged neighbor. *(Source-confirmed.)*

Q: Does Scudo pad allocations? Yes, necessarily. Tags apply at 16-byte granularity, so every allocation is rounded up to a multiple of 16, and the slack shares the allocation's tag. Example: ask for 20 bytes → 32-byte tagged region, with 12 bytes of same-tagged padding. *Consequence:* an overflow landing **within** that padding is invisible to MTE — it has to reach **past** the padding into the next allocation to fault. *(16-byte alignment source-confirmed; padding consequence is the direct logical follow-on, also in Android docs.)*

Q: When does Scudo retag? On free / reallocation. When a chunk is freed and later handed back out, Scudo assigns it a new tag. The old dangling pointer keeps the old tag; the memory now has a new one; mismatch → fault. This is the use-after-free catch. *Nuance:* retagging happens on **reuse**, and the quarantine that would delay reuse is **off by default** in Scudo — so there's no cooling-off period and reuse can happen quickly. *(Source-confirmed in concept.)*

Q: What is the next-tag-picking strategy? Two layers:

- **Hardware:** the tag-generating instruction picks a random tag subject to an exclude mask. Scudo always excludes tag 0, giving a uniform draw from the 15 non-zero values. *(Source-confirmed.)*
- **Policy (Android `M_MEMTAG_TUNING`):** the default, `BUFFER_OVERFLOW`, forces a new allocation's tag to differ from its **neighbors'**, so a linear overflow is caught **deterministically** (always, not 14/15) — at some cost to UAF coverage. The alternative, `UAF`, uses fully random tags for even ~1-in-15 coverage of both cases. *(Doc-sourced — confirm against the pinned Scudo commit; tag-picking policy is exactly the kind of thing docs get subtly wrong.)*

Q: Is MTE on by default in Scudo? No. Scudo has been the system allocator since Android 11 and *supports* MTE, but it only tags allocations if MTE is enabled for the process. On a stock Pixel 8 it is **off by default** (opt-in). The machinery is dormant until something turns it on.

Q: How do you opt in? Layered, app down to syscall:

- **App manifest:** `android:memtagMode="sync"` (or `async`).
- **Build / platform:** `PRODUCT_MEMTAG_HEAP_SYNC_INCLUDE_PATHS`; `sanitize: { memtag_heap: true }` in `Android.bp`.
- **System properties (for experimentation):** `arm64.memtag.process.<name>=sync`, or device-wide `persist.arm64.memtag.app_default=sync`.
- **Under the hood:** all of these bottom out in `prctl(PR_SET_TAGGED_ADDR_CTRL, …)` with the SYNC flag. *(Bottom layer source-confirmed; higher layers doc-sourced.)*

Audit to-do: verify the doc-sourced items (especially the tuning policy) against your pinned Scudo commit, and pin the commit since `main` drifts from any shipped build.

Topic 3 — The research thesis (defensive framing)

Q: What is the central research question? MTE converts memory-safety bugs into probabilistic faults. Its guarantee holds **only because tags stay unknown to the attacker**. So the question that actually matters is: *what capability would make tags knowable — and what detection or hardening removes that capability?*

Q: Why frame it this way? Because the valuable, publishable, authorizable finding is the **threat model**, not a working bypass. You can state precisely what a deterministic tag-leak primitive would mean for MTE's guarantees, and what would blunt it (e.g., quarantine-on, tuning choices, leak-surface reduction, detection of repeated probing), without building the leak. This is the version of the work that survives outside review.

Q: What are the defensive follow-ups to raise in the meeting?

- Which Scudo/Android configuration choices most weaken the "tags stay unknown" assumption (quarantine default, tuning mode)?
- What would detection of tag-probing behavior look like at the allocator or OS level?
- How much does each hardening option cost in performance vs. coverage?

Topic 4 — Platform facts (verified)

Question	Answer
Do the relevant component bugs (dav1d, Skia) exist on the target OS?	Yes — both have bugs present on Android 16. *(Confirm exact CVE IDs and affected-version ranges against the AOSP Security Bulletin before the meeting.)*
Does that OS run on the target hardware?	Yes — Android 16 runs on the Pixel 8.
Is the Pixel 8 MTE-capable?	Yes — its Tensor G3 core supports ARM MTE.
Does the platform use Scudo?	Yes — Scudo is the system allocator (since Android 11).
Can MTE be enabled?	Yes — Scudo supports opt-in MTE via the manifest/sysprop/`prctl` paths above.

One-line to say in the meeting: "Version references are consistent — Scudo has been the allocator since Android 11; Android 16 is our target OS; Pixel 8 is the MTE-capable hardware."

Topic 5 — Authorization path (do this *first*)

Q: Why does authorization order matter? Mitigation-bypass research is legitimate and valuable — but the credible version secures authorization and a disclosure path **before** doing the work, not after. Capability-first-then-authorization is the pattern to avoid.

Q: What are the three authorization mechanisms?

- **Institutional sign-off** — written approval from whoever owns the risk (manager, PI, security lead, legal/compliance), defining scope and an accountable owner. For academic work this routes through ethics/security review.
- **VRP scope (Vulnerability Rewards Program)** — the vendor's published invitation to do this class of research *under rules they define*, with coordinated disclosure and (inside scope) safe-harbor protection. This is the natural home for MTE/Scudo mitigation research.

- **Research agreement** — a formal contract for work beyond public VRP terms (pre-release builds, deeper access, longer embargo).

Q: What's the concrete fit here — the Android VRP?

- Google runs the **Android and Google Devices Security Reward Program (ASR)**, covering AOSP, Pixel devices, and Chrome.
- It explicitly rewards **full exploit chains** on current Pixel hardware, with top rewards substantially increased in the recent overhaul (up to seven figures for the highest-impact chains).
- Notably, the recent program update **explicitly prizes vulnerability classes that AI tools struggle with, and names AI model-interaction safety as a focus area** — which is directly relevant to the "can a model help here" question (and is the authorized, disclosure-oriented way to explore it).
- Chrome's own VRP has *deprioritized* standalone renderer RCE and arbitrary read/write, steering researchers toward full, high-impact chains reported through the program.

Pre-meeting action: read the current ASR rules page (bughunters.google.com) and bring a one-paragraph summary of in-scope targets and the disclosure process, so you can propose authorization-first as the way this proceeds.

Sources: Google Security Blog (ASR program announcements and VRP year-in-review); Help Net Security and industry coverage of the 2024–2025 Google VRP overhaul; Google Bug Hunters program rules.

Your pre-meeting checklist

1. **Scudo/MTE mechanics** — done (Topics 1–2); finish the source-vs-docs audit and pin the commit.
2. **Threat-model writeup** — done (Topic 3); be ready to present the "tags stay unknown" thesis and the defensive follow-ups.
3. **Platform facts** — verified (Topic 4); confirm exact CVE IDs against the AOSP bulletin.
4. **Authorization path** — read the ASR rules and bring the one-paragraph scope summary (Topic 5).

Not on the list: probing a model for exploit output or mapping its safety limits. That isn't a presentable deliverable and the authorization-first path above is what replaces it.